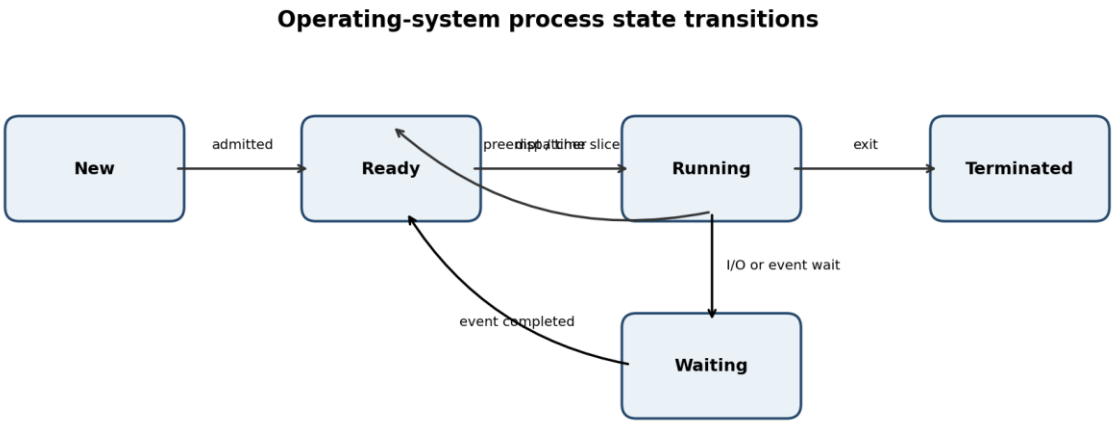
OPERATING SYSTEMS

ASSIGNMENT 2 – CO2

Process Management, Scheduling, Threads, IPC, Synchronisation and Deadlocks



Submitted in partial fulfilment of the Operating Systems course assignment

Student details	Information
Name	S. Maruthu Pandi
Register Number	192511162
Course	CSA0421 – Operating Systems
Assignment	Assignment 2 – CO2
GitHub Repository	https://github.com/Maruthupandi-maker/CSA0421-Operating-System-tree/main/CSA0421%20192511162%20Assignment%202

Q3 – THREADS

3.1 Thread definition and camera-reader application

A thread is the basic unit of CPU execution within a process. A process may contain multiple threads that execute concurrently. In the traffic-camera application, 64 cameras produce frames that must be processed. Sequential processing makes one camera wait until earlier cameras finish. A thread pool allows several frames to be processed at the same time.

3.2 Resources shared and private

Shared by threads in one process	Private to each thread
Program code and executable instructions	Thread ID and execution state
Global and heap memory	Stack and local automatic variables
Open file descriptors and process resources	Program counter and CPU registers
Address space and memory mappings	Thread-local storage
Process credentials and signal dispositions (with OS-specific details)	Scheduling information such as priority and CPU affinity

Sharing reduces memory duplication and makes communication fast, but it introduces race conditions. If two threads modify a shared counter at the same time without a lock or atomic instruction, one update can overwrite the other.

3.3 User-level and kernel-level threads

Thread model	Description	Evaluation
User-level threads	A user-space library schedules threads without requiring the kernel to know each one individually.	Fast user-space switching may be possible, but in a many-to-one model one blocking system call can block all user threads and multiple threads may not run on multiple CPU cores.

Kernel-level threads	The operating-system kernel knows and schedules individual threads.	Supports true parallel execution on multiple cores and independent blocking, but thread management and switching require kernel involvement.

3.4 Thread mapping models

Model	Mapping	Advantages	Limitations
Many-to-one	Many user threads map to one kernel thread.	Low user-space management overhead.	No true parallelism on multiple cores; one blocking call can block all threads.
One-to-one	Each user thread maps to a kernel thread.	Parallel execution and independent blocking.	Kernel resource use increases as the number of threads grows.
Many-to-many	Many user threads are multiplexed over a smaller or equal number of kernel threads.	Balances user-level flexibility and kernel parallelism.	More complex to implement and manage.

3.5 POSIX thread benchmark for a worker pool

The following program creates a selected number of POSIX worker threads. An atomic counter assigns each camera frame exactly once. Each worker performs repeatable CPU work and stores its result in a camera-specific array element. The command-line argument selects 1, 4, 8, 16 or 64 workers. It measures elapsed wall-clock time on the machine where it is executed.

```c
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <stdatomic.h>
#include <time.h>

#define CAMERAS 64
#define WORK 300000
static atomic_int next_camera = 0;
static unsigned long long result[CAMERAS];

static void *worker(void *arg) {
    (void)arg;
    for (;;) {
        int c = atomic_fetch_add(&next_camera, 1);
        if (c >= CAMERAS) break;
        unsigned long long x = (unsigned long long)(c + 1);
        for (int i = 0; i < WORK; ++i)
            x = (x * 1664525ULL + 1013904223ULL + (unsigned)i) & 0xffffffffULL;
        result[c] = x;
    }
    return NULL;
}

int main(int argc, char **argv) {
    int n = (argc > 1) ? atoi(argv[1]) : 4;
    if (n < 1 || n > CAMERAS) {
        fprintf(stderr, "usage: %s threads(1..64)\n", argv[0]); return 1;
    }
    pthread_t *th = calloc((size_t)n, sizeof(*th));
    if (!th) { perror("calloc"); return 1; }
    struct timespec a, b;
```

```
    clock_gettime(CLOCK_MONOTONIC, &a);
    for (int i = 0; i < n; ++i) {
        int e = pthread_create(&th[i], NULL, worker, NULL);
        if (e != 0) { fprintf(stderr, "pthread_create: %d\n", e); return 1; }
    }
    for (int i = 0; i < n; ++i) pthread_join(th[i], NULL);
    clock_gettime(CLOCK_MONOTONIC, &b);
    double sec = (b.tv_sec-a.tv_sec) + (b.tv_nsec-a.tv_nsec)/1e9;
    unsigned long long checksum = 0;
    for (int i = 0; i < CAMERAS; ++i) checksum += result[i];
    printf("threads=%d cameras=%d elapsed=%.6f seconds checksum=%llu\n",
           n, CAMERAS, sec, checksum);
    free(th); return 0;
}
```

Compile and run all pool sizes:

```
gcc -std=c11 -O2 -Wall -Wextra -pthread q3_threads.c -o q3_threads
for n in 1 4 8 16 64; do ./q3_threads $n; done
```

3.6 Speed-up, efficiency and why scaling stops

Speed-up S(N) = T(1) / T(N), where T(1) is elapsed time with one worker and T(N) is time with N workers. Parallel efficiency E(N) = S(N) / N. A speed-up of 4 on 8 threads gives efficiency 0.5 (50%).

Speed-up stops increasing because of the finite number of CPU cores, serial parts of the program, lock/atomic contention, memory-bandwidth limits, cache misses, thread creation and scheduling overhead. Amdahl's law states that if a fraction of a program is inherently serial, adding processors cannot accelerate that serial fraction.

3.7 Shared counter race condition

Suppose two threads execute counter = counter + 1. The operation involves reading the old value, adding one and writing the new value. Both threads may read the same old value and both write the same incremented value. One increment is lost. A mutex can protect the update, or a C11 atomic operation such as atomic_fetch_add can make the update indivisible.

3.8 Alternative: multiple processes instead of threads

Separate processes have independent virtual address spaces. This improves isolation: a memory error in one process is less likely to corrupt another. However, creating and switching processes generally has more overhead, and each process duplicates some address-space and kernel bookkeeping. Threads share memory and can combine results through ordinary shared variables protected by locks; processes must combine results using IPC such as pipes, sockets, shared memory or message queues.

Aspect	**Threads**	**Processes**
Creation and switching	Usually lighter within an existing process.	Usually heavier because an independent process and address space are managed.
Memory	Share code, heap and process resources; each thread has its own stack and registers.	Separate address spaces; COW may reduce immediate duplication after fork.
Communication	Shared data is direct but must be synchronised.	IPC is required for safe data exchange.
Fault isolation	A severe fault can affect the entire process.	Stronger isolation between workers.

Recommendation: use a pool of kernel-level POSIX threads sized near the number of CPU cores for CPU-bound frame processing. Use a bounded input queue to absorb camera bursts. Separate processes may be used for independent camera services or untrusted decoder components when stronger fault isolation is more important than communication overhead.